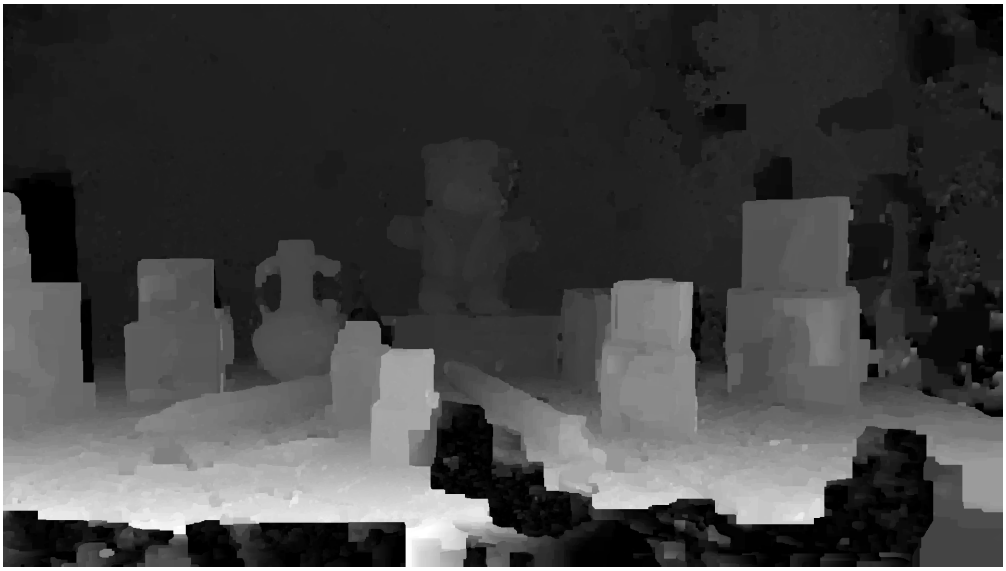
Université Libre de Bruxelles

INFO—H503

GPU computing



CUDA Planesweep Project

Antoine Berthion — 566199

May 23, 2026

Supervised by:
Bonatto Daniele and Soetens Eline

Table of contents

1 Introduction

Depth estimation is a fundamental problem in computer vision and lies at the core of many important applications, ranging from *environment perception* in robotics to various *3D modeling* use cases. It consists of estimating the distance between objects and a camera in a given scene by exploiting images captured from **different viewpoints**.

In this report, we present the *Plane Sweep* algorithm, a method used to compute a **depth map**, which is a grayscale representation of the original scene where darker pixels correspond to objects that are farther away from the camera. The algorithm works by projecting the scene onto a series of **hypothetical planes** and evaluating the consistency of color matching between different camera views.

We will first discuss the theoretical foundations of the algorithm, then present possible CPU and GPU implementations, along with several optimization techniques and benchmarking results.

2 Depth Estimation

In this section, we present the problem of depth estimation from multiple camera viewpoints. We first introduce the *Plane Sweep* algorithm, which is used to construct a cost volume representing the likelihood of different depth hypotheses for each pixel. We then describe two depth extraction methods used to generate the final depth map from this cost volume, which are used in our project.

2.1 Plane Sweep Algorithm

The Plane Sweep algorithm is a method used to estimate the depth of a scene from several calibrated camera images. The main idea is to evaluate a set of hypothetical depth planes and determine, for each pixel, which depth hypothesis provides the best correspondence between different camera views.

The algorithm starts by selecting a **reference camera**. For every pixel in this reference image, a series of candidate depth planes is tested between a near plane Z_{Near} and a far plane Z_{Far}. The sampled depth associated with a plane index z_i is computed using a projective distribution in order to obtain a denser sampling for nearby objects:

$$z = \frac{Z_{Near} \cdot Z_{Far}}{Z_{Near} + \left(\frac{z_i}{N_{Planes}}\right)(Z_{Far} - Z_{Near})}$$

For each candidate depth, the corresponding 3D point is reconstructed in the reference camera coordinate system using the inverse intrinsic matrix $\mathbf{K}^{-1}$. The point is then transformed into world coordinates using the inverse rotation and translation matrices of the reference camera. Finally, this 3D point is projected into the coordinate system of another camera using its intrinsic and extrinsic parameters. See figure 1.

This reprojection process allows us to determine where a pixel from the reference image would appear in another image if the current depth hypothesis were correct. If the depth assumption is accurate, the projected pixels should correspond to the same physical point in the scene and therefore exhibit similar intensity values.

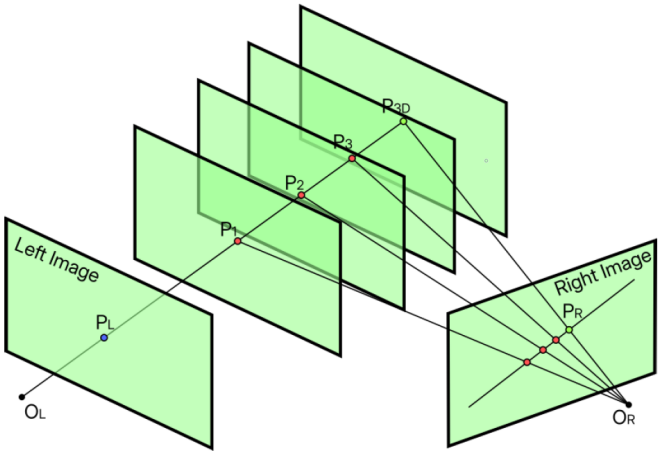


Figure 1: Planesweep algorithm

To evaluate this similarity, a matching cost is computed between the reference image and the projected image. In our implementation, we use the `Y` channel of the YUV color space and compute the average absolute difference over a square window centered around the pixel:

$$C(p, z_i) = \frac{1}{|W|} \sum_{q \in W} |I_{ref}(q) - I_{proj}(q)|$$

where W represents the matching window around pixel p.

For each depth plane, the resulting matching costs are stored inside a three-dimensional structure that will be referred to as the **cost cube** in our project. Each layer of this volume corresponds to the matching cost associated with one candidate depth plane.

Since multiple auxiliary cameras are used, the minimum matching cost across all views is retained for every pixel and every depth hypothesis. This approach improves robustness against occlusions and inaccurate matches.

2.2 Depth Extraction

Once the **cost cube** has been computed, a depth value must be assigned to each pixel to generate a **Disparity Space Image** (our grayscale image). Two different approaches are implemented in this project.

2.2.1 Argmin approach

The simplest approach consists of selecting, for every pixel, the depth plane that **minimizes** the matching cost. For each pixel (x, y), the selected depth is defined as:

$$D(x, y) = \arg\min_{z_i} C(x, y, z_i)$$

This approach is computationally efficient and easy to implement. However, because each pixel is processed independently, the resulting depth maps are often noisy or granular and may contain discontinuities or unstable regions in low-texture areas.

2.2.2 Graph-Cut Optimization

To obtain smoother and more coherent depth maps, a second approach based on graph-cut optimization is also implemented.

In this method, the depth estimation problem is formulated as an energy minimization problem composed of two terms:

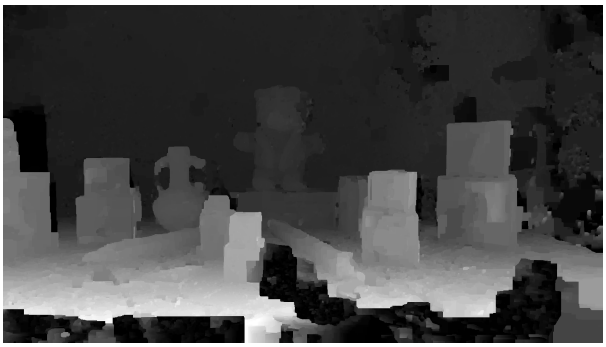
- a **data term**, corresponding to the matching cost extracted from the cost volume,
- a **smoothness term**, encouraging neighboring pixels to have similar depth values to smoothen the image.

A graph is constructed where each pixel corresponds to a node connected to a source and a sink. Additional edges are added between neighboring pixels in order to penalize abrupt depth variations. The optimal labeling is then obtained by solving a *maximum-flow/minimum-cut* problem.

2.3 Comparison of Depth Extraction Methods

In this subsection, we compare the two depth extraction methods introduced in Sections 2.2.1 and 2.2.2. The results are obtained using a **plane sweep with 256 depth planes**.

(a) Argmin approach



(b) Graph-cut optimization

Figure 2: Comparison of depth extraction methods

As discussed previously, Figure 2a produces a noticeably more granular and noisy depth map. In contrast, the graph-cut result shown in Figure 2b is significantly smoother and more spatially consistent. However, this improvement comes at a much higher computational cost, since the graph-cut method requires solving a hard optimization problem.

3 Parallelization

In this section, we present the GPU implementation of the Plane Sweep algorithm, focusing on how the computation is parallelized across depth planes, pixels, and source cameras. We also briefly discuss the parallelization strategy used for the argmin-based depth extraction method introduced in Section 2.2.1.

3.1 Parallelization Strategy for Plane Sweep

The Plane Sweep algorithm is particularly well-suited for parallel execution due to the independence of its core computations. Each pixel in the reference image can be processed independently, and each depth

hypothesis can be evaluated separately. This naturally leads to a three-dimensional parallel structure over image width, image height, and depth planes, using three-dimensional grids.

In the GPU implementation, each thread is assigned a specific combination of pixel coordinates (x, y) and a depth plane index z_i. This allows the algorithm to compute the matching cost for a single hypothesis in parallel with all other hypotheses across the image.

For each thread, the algorithm performs the same sequence of operations as in the CPU version: the reference pixel is first unprojected into 3D space using the camera intrinsics and the current depth hypothesis. The resulting 3D point is then transformed into the world coordinate system and reprojected into each source camera. Finally, a similarity measure is computed between the reference image and the projected source image over a local window.

Since multiple source cameras are available, each thread aggregates the matching cost across all views and retains the minimum cost. This step ensures robustness against occlusions and viewpoint-dependent errors.

The full cost volume is therefore computed in a massively parallel manner, where each thread contributes exactly one value to the 3D cost structure. This removes the need for **nested CPU loops** over pixels, depth planes, and camera views, and significantly reduces computation time.

3.2 Parallelization of Depth Extraction (Argmin)

Once the cost volume has been computed, the depth extraction step consists of selecting, for each pixel, the depth plane that minimizes the matching cost.

This operation is also highly parallelizable, since each pixel can independently search for its best depth without requiring information from neighboring pixels. In the GPU implementation, each thread is assigned a single pixel and iterates over all depth planes to find the minimum cost.

Although this step is less computationally expensive than cost volume construction, its parallel execution ensures that no sequential bottleneck remains in the pipeline, allowing the full depth estimation process to be executed efficiently on the GPU.

4 Optimizations

This section describes the main implementation optimizations used in the CUDA version of the Plane Sweep algorithm. The goal is not to change the algorithm itself, but to make it run efficiently on the GPU by reducing memory overhead, avoiding unnecessary stalls, and improving data access patterns.

4.1 Flattened data structures

One of the first changes compared to the CPU version is how data is stored. On the CPU, cameras and matrices are handled using complex C++ structures. On the GPU, these are replaced by a simple `DeviceCam` structure containing only fixed-size arrays.

This makes memory access much simpler and avoids pointer indirections, which are expensive on the GPU.

The cost volume is also stored as a single continuous 1D array instead of multiple separate structures. Each depth plane is stored one after another in memory. This makes indexing simpler and improves memory locality.

4.2 Constant memory for camera parameters

Camera parameters do not change during execution, so they are stored in constant memory on the GPU.

This is useful because constant memory is cached and works well when many threads read the same values. Since all threads use the same camera matrices during projection, this reduces repeated global memory accesses and helps avoid memory stalls.

4.3 Shared memory for image windows

The matching cost is computed over a local window around each pixel. In the CPU version, this leads to many repeated reads of the same pixels.

On the GPU, each block first loads a tile of the reference image into shared memory. Threads inside the block then reuse this data when computing the window cost.

This reduces global memory traffic and avoids slow repeated accesses, which are a common source of performance bottlenecks.

4.4 Reducing branch overhead

The cost computation includes several boundary checks (image limits, projection validity, etc.). On a GPU, too many branches can slow down execution because threads in the same warp may take different paths.

To limit this, invalid projections are handled early, and most checks are simplified so that the main inner loop stays as uniform as possible across threads.

4.5 Memory layout of the cost volume

The cost volume is stored in a depth-major linear format, meaning that all values for a given depth plane are contiguous in memory.

This makes later processing (especially argmin depth extraction) simpler and faster, since each thread can scan memory sequentially when searching for the best depth.

4.6 Separate argmin kernel

Depth extraction is done in a separate kernel after the cost volume is computed.

Each thread is responsible for one pixel and loops over all depth planes to find the minimum cost. Splitting this step avoids adding extra complexity inside the main sweep kernel and keeps both stages simpler and more efficient.

4.7 Summary

Overall, the optimizations mainly focus on:

- simplifying data structures for GPU memory,
- reducing global memory traffic,
- improving reuse of frequently accessed data,
- and keeping execution paths as uniform as possible.

These changes help reduce memory stalls and make better use of GPU bandwidth without changing the core Plane Sweep algorithm.

5 Experimentation

...

6 Use of LLMs

In this brief section, we discuss the use of large language models (LLMs) in the context of this project. No LLM was used for the implementation or for understanding the project itself. However, some documentation in the project and this report were reviewed for syntax and grammar by DeepL as well as a GPT model. It should be noted that none of the information contained in this report was generated by anyone other than the author.

7 Conclusion

...

References

[1] Richard Szeliski. *Image Formation.* Computer Vision Algorithms and Application, `Springer`, chap. 2, 2011.

[2] Richard Szeliski. *Stereo Correspondance.* Computer Vision Algorithms and Application, `Springer`, chap. 11, 2011.